

A I 时代的编程指南

读懂语言



Charles Tao 著，与 Claude 协作完成

books.stickmancharles.com · v1.0

正文 CC BY-NC-ND 4.0 · 代码 MIT

署名: Charles Tao, 《AI 时代的编程指南》, books.stickmancharles.com

AI 时代的编程指南 · 第三册《读懂语言》

读懂 AI 写下的每一个文件——然后，敢改其中一行。

作者：Charles Tao（与 Claude 协作完成） 版本：v1.0 · 2026 年 7 月 · 系列共五册，本册难度
★★★

小序

这一册的目标只有一行：读得懂、改得动、认得出问题。不是学会从零写代码——那件事 AI 做得很好——而是让你打开项目里任何一个文件时，不再是一片天书。

读法和前两册略有不同：这一册要带着书去现场。每章讲一门语言，配一张“认脸卡”（三十秒认出这是谁）和一块真实标本。标本全部来自我们自己的东西：第二册综合演练留下的 `hello-web`（当时就说过，它的 `index.html` 会在本册上解剖台）、docs 网站的原型、我们的主仓库、ForgeCard 的树莓派代码。所以读每一章时，最好旁边开着 VS Code。

开读前说一句安神的话。机器语言看起来吓人，其实比任何人类语言都好学：没有多义词，没有言外之意，没有不规则动词。每个符号只有一个意思，每条规则永远成立。你觉得难，只是因为还没人把它们一个个正经介绍给你。这一册就是那场介绍。

目录

1. 总论：AI 时代怎么“学”一门语言
2. HTML：骨架语言
3. CSS：样式语言
4. JavaScript：动作语言
5. TypeScript：带“形状检查”的 JS
6. JSON 与配置文件家族
7. Markdown 与 MDX：文档语言
8. SQL 再进一步：读懂一条真正的查询
9. 我们栈里的其他面孔：Python、Aiken、bash

10. 实战：挑一个真实文件，和 AI 一起逐行读通

附：本册新词

第 1 章 总论：AI 时代怎么"学"一门语言

过去学一门编程语言，意味着背语法、做习题、亲手写出每一行——因为产出代码的是你。现在产出代码的是 AI，你的角色是验收（第二册整册都在练这件事）。角色变了，要学的东西就变了：审查的前提是看得懂。你每天在差异视图里看红绿，如果那些行本身是天书，审查就只是仪式。

所以本册给每门语言定一把三层的标尺。第一层，读得懂：拿到一个文件，说得它在干什么、每一块各是什么角色。第二层，改得动：能安全地做小改——换句文案、调个数字、加一行。第三层，认得出问题：哪里可疑、哪里和需求不符、哪里踩了你的红线。本册把每门语言都推到第一层，把几门重点语言推到第二、三层。

三个通用心法，先立在这里。****其一，认脸先于认字。****每门语言有自己的面相：HTML 满眼尖括号，CSS 满眼冒号分号花括号，JS 满眼 `const` 和 `=>`，JSON 满眼引号。文件后缀是身份证，面相是长相——两样对上，你已经知道自己面对的是谁。****其二，角色先于语法。****第一册第 1 章说过：网页 = HTML（骨架）+ CSS（皮肤）+ JS（动作）。拿到任何文件先答一道选择题——它是骨架、皮肤、动作、数据，还是文档？答对这题，文件已经读懂一半。****其三，读真文件，不读教程。****教程里的例子是无菌室，你的项目才是真实世界。读法三步：先看整体形状（分几大块），再认每块的角色，最后挑最不懂的一小块逐行问 AI——它是世上最有耐心的陪读。

关键领悟

语法可以忘，角色不能忘。本册每一章真正要你带走的，是"这门语言负责回答什么问题"。

试一试

打开我们的主仓库，数一数一共出现了几种文件后缀，给每一种预先猜一个角色，把猜测记下来。读完本册回来对答案——这份错题单比任何笔记都值钱。

第 2 章 HTML：骨架语言

第二册综合演练里，你让 Claude 生成了一个最小的 `index.html`。现在把它请上解剖台。你的版本细节可能不同，骨架一定是这样：

`index.html`

```
<!DOCTYPE html>
<html lang="zh">
  <head>
    <title>Charles 的主页</title>
  </head>
  <body>
    <h1>你好，世界</h1>
    <p>这是我从零上线的第一个网站。</p>
    <a href="https://github.com/charlestao">我的 GitHub</a>
  </body>
</html>
```

读懂一切 HTML，只需要三条规则。规则一：标签（Tag）。`<p>` 开、`</p>` 关，一对标签包住一段内容，名字就是身份：`h1` 大标题、`p` 段落、`a` 链接、`img` 图片、`button` 按钮、`div` 无名的通用盒子。少数标签自己包自己（`<img>`、`<br>`），因为它们没有“内容”可包。**规则二：嵌套成树。**标签里套标签，整个文件是一棵树——`html` 是树根，下分 `head` 和 `body`，一路分叉下去。缩进就是层级的可视化。你按 F12 在 Elements 面板里看到的正是这棵树；程序员说的 DOM，就是这棵树在浏览器内存里的活体。规则三：属性（Attribute）。开标签里的 `名字="值"` 是元素的设置项：`href` 是链接去哪，`src` 是图片在哪，`id` 是唯一编号，而 `class` 是给 CSS 认的名牌——下一章的主角，先记住它。

再看 `head` 和 `body` 的分工：`head` 里是给机器看的说明（标题、编码、要加载哪些样式文件），`body` 里才是给人看的内容。页面上看得见的一切都住在 `body`。

最后一个预告：我们自己的项目里其实很少见到裸的 `.html` 文件，因为 React（第一册第 2 章）把页面拆成了组件，你会在 `.tsx` 文件里看到“长在 JS 里的 HTML”——模样几乎一样，认出来就行，第 5 章正式介绍。

认脸卡

后缀 `.html`；满眼尖括号；标签成对；缩进成树。

关键领悟

HTML 只回答一个问题——页面上有什么、谁包着谁。它不管好不好看（那是 CSS 的事），也不管点了会怎样（那是 JS 的事）。

试一试

打开你的 `index.html`，挑三个标签说出各自的身份和属性的含义。然后故意删掉一个关闭标签，刷新浏览器看页面如何错乱——再改回来。破坏一次再修复，比读十遍都记得牢。

第 3 章 CSS：样式语言

CSS 的整个世界，就是下面这个句式重复几百遍：

□ CSS

```
.hero-title {  
  color: #ffffff;  
  font-size: 38px;  
  margin-bottom: 14px;  
}
```

读法：选择器（选中谁）+ 花括号里一串 **属性：值；**（把它们改成什么样）。这一条读作："所有戴着 `hero-title` 名牌的元素，字白色、38 号、底下留 14 像素。"

选择器最常见三种：标签名（`p` 选中所有段落）、`.class`（选中戴这块名牌的，实战中最常用——名牌就是上一章 HTML 里的 `class` 属性，两章在这里握手）、`#id`（选中那个唯一编号）。组合选择器从右往左读：`.nav a` 是"链接，且住在 `.nav` 里面的"。

然后是本章最值钱的概念：盒模型（Box Model）。每个元素都是一个矩形盒子，从里到外四层：内容 → `padding`（内边距，盒子内的留白）→ `border`（边框）→ `margin`（外边距，和邻居的距离）。界面上一切"间距不对"的问题，九成出在 `padding` 和 `margin` 之间。F12 点选任何元素，样式面板里那张彩色靶心图就是它的盒模型，实时可看、现场可改。

还有一个现象要提前解释，免得你打开我们的项目一头雾水：为什么代码里几乎看不到上面那种 CSS，反而满屏是这个——

□ jsx

```
<div className="flex items-center gap-4 text-sm text-gray-400">
```

这是 Tailwind：一本预制的 class 字典。每个词对应一小条现成的 CSS——`flex` 是弹性排列，`gap-4` 是间距，`text-sm` 是小号字，`text-gray-400` 是灰色。传统写法是"起个名牌，去 CSS 文件里写规则"；Tailwind 是"直接把规则的名字挂在名牌位上"。它省去了起名和切换文件，AI 生成起来极其顺手，代价是 HTML 看起来很吵。读法不变：一个词就是一条"属性: 值"，不认识

的词问 AI，一问一个准。顺带一提：我们主站那份手写的 `style.css` 是传统派，项目里的 React 界面是 Tailwind 派——两种你都会撞见，两种都要认得。

认脸卡

后缀 `.css`，满眼 `{ 属性: 值; }`；或者根本没有独立文件，样式藏在 `className` 的一长串单词里（Tailwind）。

关键领悟

CSS 只回答"长什么样、摆在哪"。读它的钥匙是选择器——先搞清这条规则管到谁头上，再看它改了什么。

试一试

F12 打开我们自己的网站，点选一个按钮，在 Styles 面板里现场改它的 `color` 和 `padding`——所见即所得，刷新就还原，零风险。再从项目里挑一段 Tailwind class，逐词让 AI 翻译成"属性: 值"。

第 4 章 JavaScript：动作语言

三兄弟里，JS 是唯一的“编程语言”——有逻辑、会判断、能记事。先把四块积木认全：

📄js

```
const price = 100;                // 变量：给一个值起名字
function applyDiscount(p) {        // 函数：打包一段动作，随叫随用
  if (p > 50) {                    // 判断：满足条件才做
    return p * 0.9;
  }
  return p;
}
const items = ["卡片", "凭证"];    // 数组：一列东西
const user = { name: "Roy", age: 47 }; // 对象：一组「名字：值」
```

变量用 `const`（之后不再改）或 `let`（还会变）声明——AI 几乎总用 `const`，图的是安全。函数是“输入 → 加工 → `return` 输出”；它有个现代简写叫箭头函数：`const f = (p) => p * 0.9`，意思完全一样，只是换了件衣服——你在 AI 写的代码里看到的满屏 `=>`，都是它。对象与数组你其实见过：它们就是 JSON 的活体（第 6 章会正式认这门亲戚），`user.name` 读作“user 的 name”。

第四块积木是浏览器 JS 的心跳：事件（Event）——“等到某件事发生，就执行这段代码”。别处不用找例子，我们 docs 原型里的复制按钮就是：

📄js

```
btn.addEventListener('click', () => {
  const code = btn.parentElement.querySelector('code').textContent;
  navigator.clipboard?.writeText(code);
  btn.textContent = '已复制';
  setTimeout(() => btn.textContent = '复制', 1600);
});
```

逐行读：等这个按钮被点 → 找到它旁边代码块里的文字 → 写进剪贴板（`?.` 读作"如果它存在的话就"）→ 把按钮文案改成"已复制" → 1.6 秒后改回来。五行，一个完整的小剧情。你上周还只是"用过"这个按钮，现在你读懂了它。

两个补充认知。 **其一，同一门语言，两个住处。 **浏览器里的 JS 摸得到页面（有 `document`、有事件）； `Node.js` 则是装在电脑或服务器上的 JS 运行环境，没有页面，但能读文件、开服务器。第一册讲的前端与后端，在 JS 世界里常常是同一门语言两头跑——这正是 `npm` 的包前后端通用的原因。 **其二， `await` 认脸即可。 **见到 `await fetch(...)`，读作"去网上取数据，等它回来再继续"。网络请求都要等， `await` 就是那个"等"字——第一册第 1 章的请求与响应，写进代码里就长这样。

认脸卡

后缀 `.js`；满眼 `const`、`function`、`=>`、`if`；行尾常有分号。

关键领悟

读 JS 不要逐字抠语法，先找三样东西——数据在哪（变量）、动作在哪（函数）、什么时候触发（事件或调用）。三样找齐，这个文件的剧情就通了。

试一试

打开 docs 原型底部的 `<script>` 区，里面一共六小段功能。挑两段，逐行讲给自己听；讲不通的行，原样贴给 AI 问"这行为什么需要存在"。

第 5 章 TypeScript：带"形状检查"的 JS

先看一眼，消除恐惧：

□ts

```
type Certificate = {  
  id: string;  
  owner: string;  
  mintedAt: number;  
};  
  
function verify(cert: Certificate): boolean {  
  return cert.id.length > 0;  
}
```

现在做个实验：把所有冒号后面的标注和整个 `type` 块用手遮住——剩下的就是你上一章刚认识的 JS。这就是读 TS 的口诀：冒号后面是"形状说明"，不是逻辑。先遮住冒号读逻辑，再回头看形状。

形状说明在说什么？提前申报每个变量、每个参数长什么样：`string` 是文本，`number` 是数字，`boolean` 是真假，而 `Certificate` 是我们自定义的形状——"一张凭证必须有文本的 `id`、文本的 `owner`、数字的 `mintedAt`"。像海关申报单：过关前先 declare 箱子里装什么，机器一扫，不符就报警。你在 VS Code 里见过的红色波浪线，多半就是这台安检机在喊："形状不对。"

为什么 AI 偏爱 TS，为什么我们的项目几乎全是 `.ts` 和 `.tsx`？两个原因。其一，错误在写的时候就被抓，不用等跑起来才炸——第一册第 11 章说过"本地被宽容、线上被严查"，TS 就是把严查提前到了打字的那一秒。**其二，类型是给 AI 的说明书。**AI 读到 `Certificate` 的定义，就确切知道这份数据有什么、没有什么，幻觉大减。类型越全的代码库，AI 改起来越稳——这也算一种"给 AI 立规矩"（第二册第 11 章），只是写法是代码而不是文档。

再认一个高频面孔：`.tsx`——TS 加上 JSX（第 2 章预告过的"长在 JS 里的 HTML"）。我们项目里一个典型的组件文件是三段式：顶部一排 `import`（借来的积木），中间一个 `type` 定义

props（这个组件接受什么参数），下面一个函数 `return` 出一段"HTML"。认出这个三段式，React 组件就不再神秘。

TS 的报错啰嗦但诚实，核心句式永远是：Type A is not assignable to type B ——"你给的形状 A，塞不进要求的形状 B"。贴给 AI 之前自己先猜一次：是哪两个形状没对上？

认脸卡

后缀 `.ts` / `.tsx`；满眼冒号标注、`type`、`interface`；红波浪线是它的声音。

关键领悟

TS 不是一门新语言，是 JS 加了一层"出错前就报警"的安检。读它，遮住冒号；改它，让逻辑和形状保持一致——改了其一，另一半跟上。

试一试

打开项目里任意一个 `.tsx`，找到 `type` 或 `interface` 块，用一句话说出"这个组件要吃什么参数"。再故意把某个文本值改成数字，看红波浪线几秒内亮起——安检正在为你上班。

第 6 章 JSON 与配置文件家族

JSON 在第一册第 4 章露过面（软件之间传的单子），这一章把它升级成“读写自如”。全部规则只有四条：`{ }` 包一组“名字: 值”（对象），`[ ]` 包一系列东西（数组），名字必须带双引号，条目之间用逗号——且最后一项后面不许留逗号，这是手改 JSON 时最经典的翻车点。值只有六种可能：文本、数字、true/false、null、另一个对象、另一个数组。就这些，没了。

拿真实标本练手——`package.json`，第二册第 10 章的“购物清单加快捷按钮”，现在你能逐行读了：

`package.json`

```
{
  "name": "hello-web",
  "scripts": {
    "dev": "vite dev",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^18.3.1"
  }
}
```

连 `^18.3.1` 头上那顶小帽子都有意思：它读作“18 这个大版本内，允许自动更新”——第一册第 10 章讲的 semver，在这里现形。

家族里还有两位常客要认脸。YAML（`.yaml` / `.yml`）：用缩进代替花括号的“晚礼服 JSON”，`.github/workflows/` 里的自动流水线穿的就是它——缩进即层级，冒号后面是值，`-` 开头是列表项。它优雅，但有洁癖：缩进错一格，整个文件瘫痪，改它时对齐就是命。`.env`：最朴素的一族，`名字=值`，一行一条，没有引号没有括号——但纪律最重，第一册第 9 章说过：永不进 Git。

这一家子有条共同的天性，值得单独说破：配置文件没有逻辑，只有事实。它们自己不“做”任何事，只是被程序读取的设置表。所以它们是全项目里最适合你亲手改的文件——改错了程序会

明确抱怨，不会静悄悄坏掉。当然，改之前依然先 commit（第二册的存档习惯，任何时候都成立）。

认脸卡

花括号加双引号 = JSON；缩进加冒号、没有括号 = YAML；等号一行一条 = env。

关键领悟

从配置文件开始建立"我敢改文件"的手感——没有逻辑、改动可见、错了就报，是最安全的第一块陆地。

试一试

打开我们项目的 `package.json`，把 `scripts` 里每一条翻译成成人话。再打开 `.github/workflows/` 里任意一个 yml，只回答一个问题：这条流水线在什么时候被触发？（提示：找 `on:` 那一行。）

第 7 章 Markdown 与 MDX：文档语言

先说一个你可能没意识到的事实：这门语言你已经会了。这套书的五个文件全是 Markdown 写的，你读了三册，等于泡在语料里几星期。语法十分钟能列完：`#` 的个数是标题层级，`**`包住`**` 是粗体，`-` 起头是列表，`[文字](网址)` 是链接，三个反引号围出代码块，`|` 画表格，`---` 是分隔线。就这些。

一门这么简单的语言，为什么统治了 README、CLAUDE.md、AI 的回答和我们这套书？三个理由。它是纯文本——所以 Git 能对它做红绿差异，文档从此和代码享受同等待遇：一样存档、一样 PR、一样审查（第二册的整套流程对文档同样生效）。它不渲染也能读——就算没有任何工具，打开就是干净的文字。它到处通用——GitHub、编辑器、AI 对话框，走哪都认。“文档进 Git”是现代工程的默认姿势，Markdown 就是这个姿势的载体。

接着认识它的进化形态：MDX = Markdown + 可以嵌入 React 组件。在 `.mdx` 文件里，大部分内容还是普通 Markdown，但你可以写 `<TryIt>...</TryIt>` 这样的自定义标签，让文档里长出交互块。我们 docs 网站上的“关键领悟”卡片、悬停出释义的术语，将来大概率就是这么实现的：内容仍是 Markdown，特殊块是组件。认脸有个小窍门（也是 JSX 的通行规矩）：大写开头的标签是组件，小写开头的是普通 HTML。

最后认一个文档的“名片”：front matter——文件顶部两条 `---` 夹着的一小段 YAML（上一章刚学，立刻用上）：标题、日期、标签。docs 框架靠它生成侧边栏、排序和页面信息。你以后拆书成站，每一章头上都会有这么一张名片。

认脸卡

后缀 `.md` / `.mdx`；`#` 开头的标题；`.mdx` 里偶尔冒出大写开头的标签。

关键领悟

Markdown 是你和 AI 的母语交汇点——你用它写需求、写 CLAUDE.md、写红线，AI 用它回答你。把 Markdown 写清楚，就是把给 AI 的输入写清楚。

试一试

把你最近征服的一个新词，亲手按格式补进第一册附录 A（粗体术语加一句解释），commit，写清存档说明。这是你第一次以作者身份修改这套书——从此它真的是"活的"了。

第 8 章 SQL 再进一步：读懂一条真正的查询

第一册第 3 章给了你四个动词，这一章读一条带筋骨的真查询——它长得像我们 Supabase 里会出现的样子：

□sql

```
SELECT certificates.id, users.email
FROM certificates
JOIN users ON users.id = certificates.owner_id
WHERE certificates.status = 'minted'
ORDER BY certificates.created_at DESC
LIMIT 20;
```

SQL 有个反直觉的秘密：读它不从第一行读，从 **FROM** 读。顺序是——**FROM**：从 `certificates` 表出发；**JOIN ... ON**：把 `users` 表按对应关系拼过来（凭证的 `owner_id` 对上用户的 `id`——这种“表与表之间的工号”叫外键，第四册讲怎么设计，这册先会读）；**WHERE**：只留状态是 `minted` 的行；**ORDER BY ... DESC**：按创建时间倒序，新的在前；**LIMIT 20**：只要前二十条；最后才轮到 **SELECT**：从结果里端出哪几列。整句人话：“找出最近铸造成功的二十张凭证，附上主人的邮箱。”

JOIN 的直觉不难：两张 Excel 表按“工号”对齐、拼成一张宽表。为什么当初要拆成两张？因为用户信息只该存一份——用户改邮箱，只改一处，所有凭证自动“知道”。这条不重复原则是数据库设计的地基，第四册展开。

接下来是本章的红线雷达，请刻进反射弧：**见到 **UPDATE** 或 **DELETE** 后面没有 **WHERE**，立刻停下叫人。**没有 **WHERE**，意思是“整张表全部改掉/删掉”。AI 写的迁移脚本、清理脚本里一旦出现这个形状，无论它解释得多合理，都触发第一册红线卡上那条“直接操作生产数据库”——人是最后一道闸。

最后说说 SQL 在我们生活里藏在哪：Supabase 的 Table Editor 里每次点击背后都是一条 SQL；SQL Editor 能直接跑；第一册第 3 章讲的迁移文件，本体就是一串 SQL 存档——AI 每次“建表”产出的就是它们。从今天起，你能读它的迁移了。至于那些 **CREATE POLICY** 开头的语句

——那是 RLS 的权限规则（谁能看哪些行），也是 SQL；这册认出它是什么就行，第四册第 6 章亲手写。

认脸卡

后缀 `.sql`；关键词习惯全大写；分号收尾；全家族里英语味最重的一门。

关键领悟

读 SQL 的顺序是 FROM → JOIN → WHERE → SELECT。外加一条铁律：没有 WHERE 的 UPDATE/DELETE，永远先叫人。

试一试

打开 Supabase 的 SQL Editor，让 AI 给我们某两张表写一条只读的 JOIN 查询。先自己逐层翻译，再运行，对照结果验证理解。（只读的 SELECT 随便跑，这是安全的练习场。）

第 9 章 我们栈里的其他面孔：Python、Aiken、bash

这一章的标准降一档：只到"认识"为止——见到能认出、大意能读懂、危险能分级，不求会改。

Python (`.py`) ——ForgeCard 树莓派上跑的就是它。最大特征：缩进就是语法。别的语言用花括号表示"哪些行属于这个块"，Python 直接用缩进表示，所以它没有花括号、也基本不写分号，长相干净得像伪代码。看一段读卡循环的骨架：

□python

```
while True:
    uid = reader.read_card()
    if uid:
        print(f"读到卡片: {uid}")
        verify(uid)
```

读：永远循环（树莓派永远醒着——第一册第 5 章那台客厅小服务器的日常）→ 读卡 → 读到了就打印并验证。`print` 就是第一册第 14 章那盏最土最有效的探照灯，Python 版。

Aiken (`.ak`) ——我们链上合约的语言。两件事记牢就够。其一：它的类型检查比 TS 更严苛，因为合约部署后不可修改（第一册第 12 章的学费），错误必须在部署前死绝，所以语言本身长成了一台超级安检机。其二：文件里那个叫 `validator` 的函数是合约的门神——每笔交易来敲门，它说放行或拒绝。本册对 `.ak` 只有一个要求：见到它，红线雷达自动拉满。它属于"改动前必须人来确认"的世界，具体规矩在我们的工程护栏里，不在这本书里。

bash 脚本 (`.sh`) ——把第二册第 4 章的命令串成一个文件，一行一条，自动化的最小形态。第一行 `#!/bin/bash` 叫 shebang，意思是"本剧本由 bash 来演"。点开项目里的 `deploy.sh`、`setup.sh` 这类文件，你会惊讶地发现：全是认识的命令在排队。

收个总账。语言们长相各异，骨架却相通：变量、函数、判断、循环，四大件人人都有。在任何陌生语言里先找这四件，第一层理解立刻建立——这就是"学第二门语言比第一门快十倍"的原因，而你现在已经在学第七门了。

认脸卡

缩进当语法、无分号 = Python; `validator` 加满屏类型 = Aiken, 红线拉满;
`#!/bin/bash` 开头、全是命令 = bash 脚本。

关键领悟

对这三门的要求是三十秒答出三问——什么语言? 大概在干什么? 危险几颗星? (`.py` 平常心, `.sh` 看内容, `.ak` 永远满星。)

试一试

打开 ForgeCard 仓库里任意一个 `.py`, 只指认四大件——变量、函数、判断、循环各一处。再打开一个 `.ak`, 什么都不改, 只把 `validator` 的名字找出来, 然后轻轻关上文件。

第 10 章 实战：挑一个真实文件，和 AI 一起逐行读通

毕业课不讲新语言，讲一套陪读流程。六步，走完一次算毕业；以后每月一次，读的能力就会一直长。

1. 选材。挑一个你项目里"每天路过但从没进去过"的文件。首选五十到一百五十行、角色清晰的：一个 React 组件、一个 API 函数，或者——最合适的毕业标本——我们的 `docs-prototype.html`：一个文件集齐了本册第 2、3、4 章（骨架、皮肤、动作三位一体）。
2. 冷读五分钟。不查不问，写下三句话：它是什么语言、什么角色？分几大块？我最看不懂的是哪几行？先猜后问，猜过的才记得牢——第一册第 14 章"先猜再对答案"，原理相同。
3. 让 AI 开导览。"把这个文件分成几个功能块，每块一句话概括，先别逐行。"先看地图，再进街巷——你已经习惯这个顺序了。
4. 逐块细读。对最不懂的那块下钻："逐行解释，每一行为什么需要存在。"追问时用这把最锋利的刀："删掉这一行，会发生什么？"——它逼出的答案，比"这行是什么意思"深十倍。
5. 动一下。做一个五分钟级的小改：换句文案、调个颜色、改个数字。跑起来验证（第一册第 14 章金字塔），commit（第二册第 5 章）。读懂的标志不是能解释，是敢改。
6. 收获入库。新词补进附录 A（上一章你已经会以作者身份改它了）；如果这个文件值得，给它在 CLAUDE.md 里记一行"它是干嘛的"——你的读书笔记，同时也是给 AI 的地图（第二册第 11 章）。

到这里，三册合龙：第一册教你认地图上的地名，第二册教你握方向盘，这一册教你读路上每一块路牌。你已经可以在这个世界里自己开车了。下一册，我们下车——打开引擎盖，看看四大件里面到底是怎么转的。

关键领悟

"读懂"的终点不是解释给自己听，而是敢做一个小改动，并亲手验证它。从解释到敢改之间隔着的那点信心，就是这一册想交给你的全部东西。

试一试

本周内完整走一次六步。把你选的文件名和三句冷读笔记发给爸爸，让他也走一次，互相当读者——两个人读同一个文件，看见的盲区不一样。

附：本册新词

- 标签 (Tag) / 属性 (Attribute) / DOM: HTML 的积木 / 积木上的设置项 / 那棵树在浏览器内存里的活体。
- 选择器 (Selector) : CSS 的"选中谁", `.class` 最常用。
- 盒模型 (Box Model) : 内容 → padding → border → margin, 每个元素都是套娃矩形。
- 层叠 (Cascade) : 多条规则撞车时, 更具体、更靠后的赢。
- Tailwind: 预制 class 字典, 把样式直接挂在 HTML 的名牌位上。
- 箭头函数 (`=>`) : function 的现代简写。
- 事件 (Event) / `addEventListener`: "等某事发生, 就执行这段"。
- Node.js: 让 JS 离开浏览器、跑在服务器上的运行环境。
- `async / await`: "去取数据, 等它回来再继续"。
- 类型 (Type) / 类型标注: 变量的形状申报单; 冒号后面那部分。
- JSX / `.tsx`: 长在 JS/TS 里的 HTML; React 组件的家。
- YAML: 用缩进代替花括号的配置语言, 流水线的礼服; 缩进即命。
- `front matter`: 文档顶部两条 `---` 夹着的名片。
- MDX: Markdown + 可嵌组件, docs 网站的语言; 大写开头 = 组件。
- JOIN / 外键 (Foreign Key) : 把两张表按"工号"拼起来 / 那个工号本身。
- `validator`: Aiken 合约里的门神函数, 放行或拒绝每笔交易。
- `shebang` (`#!`) : 脚本第一行, 声明"本剧本由谁来演"。